
Concurrencia y Paralelismo

Bloque I: Concurrencia

Julio 2025

1. Cola de espera [2 puntos]

Vamos a simular una cola de donde cada thread representa una persona que ocupa una posición. Cuando a una persona le toca el turno recibe el aviso de la anterior, hace la acción por la que esperaba turno, y avisa al siguiente. Cada thread solo debe despertar a uno, y no se puede usar una estructura de datos de tipo cola o lista.

Para avisar al siguiente cada thread puede guardar una condición en la estructura compartida que es usada por el siguiente thread para esperar por el aviso. Antes de esperar ese thread sustituye la condición por la suya propia para a su vez poder avisar al thread que venga después de él.

Si no hay ningún thread activo la condición debería estar a NULL.

```
struct wait_row {
    int waiting;          // Number of threads in the queue
    pthread_cond_t *c;    // should be NULL if there is no one waiting or doing the action
    pthread_mutex_t *m;
}

void person(struct wait_row *wr) {
    ...
    act();
    ...
}
```

```
int pthread_mutex_init(pthread_mutex_t *m, pthread_mutex_attr_t *attr);
int pthread_mutex_lock(pthread_mutex_t *m);
int pthread_mutex_trylock(pthread_mutex_t *m);
int pthread_mutex_unlock(pthread_mutex_t *m);
int pthread_cond_init(pthread_cond_t *c, pthread_cond_attr_t *attr);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mtx);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
pthread_t pthread_self();
```

Solución:

```
struct wait_row {
    int waiting;          // Number of threads in the queue
    pthread_cond_t *c;    // should be NULL if there is no one waiting or doing the action
}

void person(struct wait_row *wr) {
    pthread_cond_t next;
    pthread_cond_t *cur;
    pthread_cond_init(&next, NULL);
    pthread_mutex_lock(wr->m);
    if(wr->c != NULL) {
        waiting++;
        cur = wr->c;
        wr->c = next;
        pthread_cond_wait(cur, wr->m);
        waiting--;
    }
    pthread_mutex_unlock(wr->m);
    act();
    pthread_mutex_lock(wr->m);
    if(wr->waiting > 0) pthread_cond_signal(wr->next);
    else wr->c = NULL;
    pthread_mutex_unlock(wr->m);
}
```

2. Barrera de emparejamiento [1.5 puntos]

Implemente, utilizando los mutex de la librería pthread, una barrera que empareje a los threads de dos tipos A y B en orden de llegada. Esto es, el primer thread en llamar a `pair_barrier_a` y el primer thread en llamar a `pair_barrier_b` se emparejan, y así sucesivamente. Si un thread llega a la barrera y no hay threads del otro tipo tiene que esperar. Las funciones `pair_barrier` deberían devolver el id del thread con el que se ha emparejado.

Para la solución puede usar la cola ya implementada en las funciones `q_init`, `q_insert` y `q_remove`

```
typedef {  
    ...  
} pair_barrier_t;  
  
void q_init(queue q);  
void q_insert(queue q, void *);  
void *q_remove(queue q); // returns NULL if the queue is empty  
  
void pair_barrier_init(pair_barrier_t *pb) {  
    ...  
}  
  
pthread_t pair_barrier_a(pair_barrier_t *pb) {  
    ...  
}  
  
pthread_t pair_barrier_b(pair_barrier_t *pb) {  
    ...  
}
```

Solución:

```
typedef {
    pthread_mutex_t m;
    queue qa, qb;
} pair_barrier_t;

typedef {
    pthread_t my_id;
    pthread_t other_id;
    pthread_cond_t c;
} thr_wait_info;

void q_init(queue q);
void q_insert(queue q, void *);
void *q_remove(queue q); // returns NULL if the queue is empty

void pair_barrier_init(pair_barrier_t *pb) {
    pthread_mutex_init(&pb->m, NULL);
    q_init(pb->qa);
    q_init(pb->qb);
}

pthread_t pair_barrier_a(pair_barrier_t *pb) {
    thr_wait_info *other_inf;
    pthread_t other_id;

    pthread_mutex_lock(&pb->m);
    if((other_inf = q_remove(pb->qb)) == NULL) {
        thr_wait_info my_inf;
        pthread_cond_init(&my_inf->c, NULL);
        my_inf->my_id = pthread_self();
        q_insert(pb->qa, &my_inf);
        pthread_cond_wait(&my_inf->c, &pb->m);
        other_id = my_inf->other_id;
    } else {
        other_inf->other_id = pthread_self();
        pthread_cond_signal(other_inf->c);
        other_id = other_inf->my_id;
    }
    pthread_mutex_unlock(&pb->m);

    return other_id;
}

pthread_t pair_barrier_b(pair_barrier_t *pb) {
    // Same as pair_barrier_a, changing qa for qb and viceversa
}
```

3. Línea de procesos [1.5 puntos]

El siguiente módulo permite crear una secuencia de procesos donde cada uno conoce el PID del siguiente.

```
-module(line).
-export([start/1, get_pids/3]).

start(N) ->
    spawn(?MODULE, init, [N-1]).

get_pids(First, A, B) ->
    ...

init(0) ->
    loop(none);
init(N) ->
    Next = spawn(?MODULE, init, [N-1]),
    loop(Next).

loop(Next) ->
    receive
    ...
    end,
    loop(Next).
```

Implemente la función `get_pids/3`, que dado el PID del primer proceso, y dos posiciones A y B en la línea, devuelve los PIDS de los procesos entre las posiciones A y B. Por ejemplo, `get_pids(First, 0, 1)` devolvería una lista con los dos primeros procesos. Puede suponer que dados N procesos en la línea, $0 \leq A \leq B < N$. Las funciones `start/1` e `init/1` no pueden modificarse. Solución:

```
-module(line).
-export([start/1, get_pids/3]).

start(N) ->
    spawn(?MODULE, init, [N-1]).

get_pids(First, A, B) ->
    First ! {get_pids_request, A, B, self()},
    receive
        {get_pids_reply, Pids} -> Pids
    end.

init(0) ->
    loop(none);
init(N) ->
    Next = spawn(?MODULE, init, [N-1]),
    loop(Next).

loop(Next) ->
    receive
        {get_pids_request, 0, 0, From} ->
            From ! {get_pids_reply, [self()]};
        {get_pids_request, 0, B, From} ->
            Next ! {get_pids, B - 1, [self()], From};
        {get_pids_request, A, B, From} ->
            Next ! {get_pids_request, A-1, B-1, From};
        {get_pids, 0, Pids, From} ->
            From ! {get_pids_reply, [self() | lists:reverse(Pids)]};
        {get_pids, B, Pids, From} ->
            Next ! {get_pids, B - 1, [self() | Pids], From}
    end,
    loop(Next).
```

EXAMEN DE CONCURRENCIA Y PARALELISMO

Bloque II: Paralelismo

APELLIDOS: _____ NOMBRE: _____

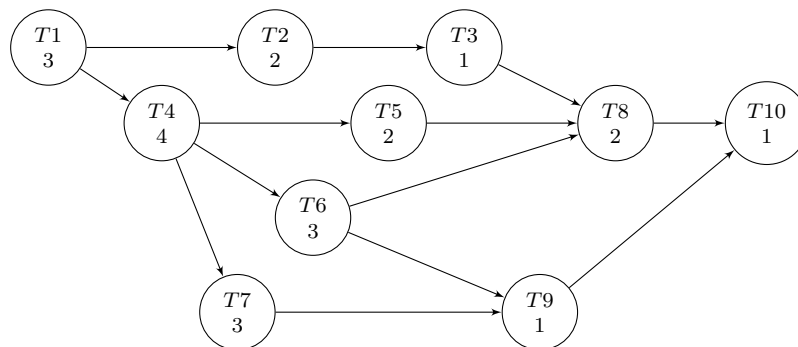
Convocatoria extraordinaria

8 de julio de 2025

AVISO: No mezcléis en el mismo folio preguntas del bloque de concurrencia y del bloque de paralelismo. Las respuestas a cada bloque se entregarán por separado.

[2.5p] 1. **Conceptos de paralelismo.**

Un programa se ha descompuesto en tareas de acuerdo al siguiente grafo de dependencias estáticas. Cada nodo tiene un peso que se corresponde con el tiempo de ejecución en segundos de dicha tarea.



Responde RAZONADAMENTE a las siguientes cuestiones:

- (a) [0.25p] Determina qué tipo de descomposición se ha utilizado.
- (b) [0.75p] Identifica el camino crítico, el grado máximo y el grado medio de concurrencia.
- (c) [0.5p] Disponemos de 4 computadores interconectados para ejecutar este algoritmo en paralelo. Cualquier comunicación entre dos computadores tiene un coste fijo de 1 segundo, pero puede solaparse con la computación de otras tareas. Determina la asignación de tareas óptima y utilizando el menor número de recursos.
- (d) [0.4p] Para la asignación escogida, calcula: aceleración, eficiencia paralela, coste y sobrecarga.
- (e) [0.6p] Considera una tarea paralelizable cuyo tiempo de ejecución depende del tamaño de los datos de entrada (N). En la siguiente tabla se muestra el tiempo que tardó la tarea (en segundos) al ejecutarla con datos de diferentes tamaños sobre diferente número de procesos. Determina si esta paralelización presenta escalabilidad fuerte y/o escalabilidad débil.

N	Procesos				
	1	2	4	8	16
5	7.0	4.0	3.0	4.0	5.0
10	11.0	6.0	6.0	7.0	7.0
20	22.0	12.0	6.0	6.0	7.0
40	43.0	21.0	11.0	5.0	8.0
80	85.0	45.0	23.0	11.0	6.0

[2.5p] 2. Diseño de algoritmos paralelos.

El siguiente programa implementa un algoritmo con un reparto de tareas dinámico en el que la función `read(int *num, int **vec)` sólo se puede ejecutar en el proceso 0 y lee de disco un vector de números enteros mayores que 0 que hay que procesar. La función recibe dos argumentos a través de los cuales devuelve el número de datos leídos y un puntero a una zona de memoria reservada con `malloc` donde los ha almacenado.

```
int main(int argc, char **argv)
{ MPI_Status s;
  int rank, size, N, *v, i = 1, z = 0;
  double r = 0, tmp;

  MPI_Init (&argc, &argv);
  MPI_Comm_rank (MPI_COMM_WORLD, &rank);
  MPI_Comm_size (MPI_COMM_WORLD, &size);

  if(!rank) {
    read(&N, &v);
    for(i=0; i < (size - 1); i++)
      MPI_Send(&v[i], 1, MPI_INT, i + 1, 0, MPI_COMM_WORLD);
    while(i < N + (size - 1)) {
      MPI_Recv(&tmp, 1, MPI_DOUBLE, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, &s);
      r += tmp;
      MPI_Send(((i < N) ? &v[i] : &z), 1, MPI_INT, s.MPI_SOURCE, 0, MPI_COMM_WORLD);
      i++;
    }
    printf("Result: %lf\n", r);
  } else {
    while(i) {
      MPI_Recv(&i, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
      if(i) {
        tmp = f(i);
        MPI_Send(&tmp, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);
      };
    }
  }

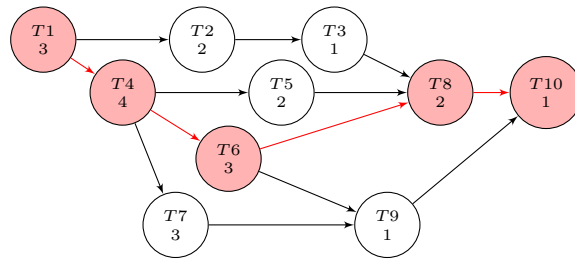
  MPI_Finalize();
  return 0;
}
```

- (a) [0.5p] Sabiendo que está garantizado que siempre se va a ejecutar con al menos dos procesos, ¿detectas algún problema potencial en la implementación? En ese caso, ¿cuál?
- (b) [2.0p] Implementa el algoritmo propuesto paralelizado pero siguiendo un reparto de tareas estático. Se recomienda **comentar y justificar** las decisiones tomadas. Para simplificar el código:
- Se puede asumir que el entorno MPI ya está inicializado y existen las variables **rank** y **size** con los valores correspondientes en cada proceso.
 - Los buffers de envío y recepción pueden solaparse.
 - Puedes definir abreviaturas, como por ejemplo MCW para `MPI_COMM_WORLD`.

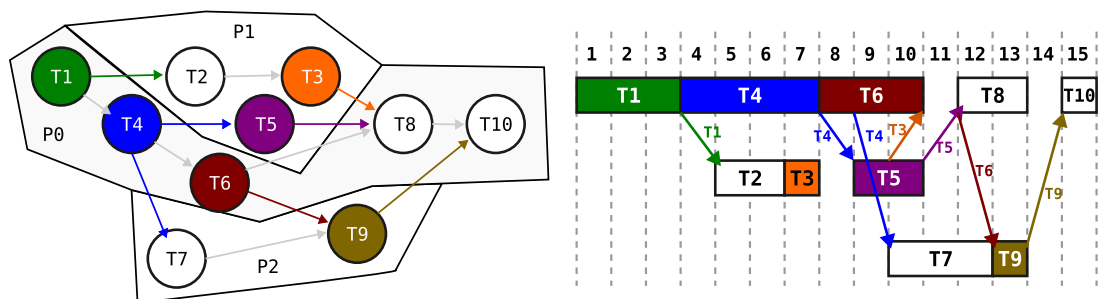

```
int MPI_Barrier(MPI_Comm comm)
int MPI_Bcast(void *buffer, int count, MPI_Datatype dt,
              int root, MPI_Comm comm)
int MPI_Scatter(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Gather(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
               MPI_Datatype dt, MPI_Op op, int root, MPI_Comm comm)
```

SOLUCIÓN

1. (a) Es una descomposici' on funcional.
- (b) El camino cr'itico se compone de las tareas T1, T4, T6, T8 y T10, y tiene una duraci' on de 13 segundos. El grado m'aximo de concurrencia es 4 y el grado medio es $22/13 = 1.69$.



- (c) Como el coste de comunicaciones es alto, interesa minimizarlas y asignar tareas consecutivas a un mismo proceso. Por ejemplo, el camino cr'itico al proceso 0. El resto de tareas hay m'ultiples formas de asignarlas y se puede conseguir el mismo reparto 'optimo con 3 procesos en lugar de 4. En el siguiente gr'afico se muestran en color las tareas que requieren comunicaciones entre procesos para identificarlas f'acilmente.



- (d) Con esta asignaci' on propuesta, la duraci' on del algoritmo es de 15 segundos, pues hay dos comunicaciones que no se pueden solapar con la computaci' on. El tiempo secuencial es la suma de la duraci' on de todas las tareas: 22 segundos.

- $A = 22/15 = 1.47$
- $Ef = 1.47/3 = 0.49 = 49\%$
- $Coste = 15 s \times 3 = 45 s$
- $Sobrecarga = Coste - T_{seq} = 45 - 22 = 23 s$

- (e) Si nos fijamos en una fila de la tabla (N constante), observamos que a partir de cierto n'umero de procesos el tiempo de ejecuci' on se mantiene o incluso empeora, por lo que podemos decir que la paralelizaci' on propuesta presenta mala escalabilidad fuerte cuando N es bajo. Sin embargo, si observamos c'omo se comporta a medida que incrementamos el n'umero de procesos manteniendo la carga por proceso constante ($p=1$ N=5, $p=2$ N=10, $p=4$ N=20, etc), el tiempo de ejecuci' on aproximadamente es constante y cercano a N/p , por lo que la eficiencia paralela es alta y confirma que la paralelizaci' on presenta buena escalabilidad d'ebil.

N	Procesos				
	1	2	4	8	16
5	7.0	4.0	3.0	4.0	5.0
10	11.0	6.0	6.0	7.0	7.0
20	22.0	12.0	6.0	6.0	7.0
40	43.0	21.0	11.0	5.0	8.0
80	85.0	45.0	23.0	11.0	6.0

N	Procesos				
	1	2	4	8	16
5	7.0	4.0	3.0	4.0	5.0
10	11.0	6.0	6.0	7.0	7.0
20	22.0	12.0	6.0	6.0	7.0
40	43.0	21.0	11.0	5.0	8.0
80	85.0	45.0	23.0	11.0	6.0

2. (a) Sí, hay un problema. El código asume que va a haber al menos `size - 1` datos a procesar, pero podría haber menos.

(b)

```
int N, *v, i, elems;
double tmp = 0., r;

if(!rank) {
    read(&N, &v);
}

MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
elems = (N + size - 1) / size;

if(!rank) {
    if(N % size) {
        v = (int *)realloc(v, sizeof(int) * elems * size);
    }
} else {
    v = (int *)malloc(sizeof(int) * elems);
}

MPI_Scatter(v, elems, MPI_INT, v, elems, MPI_INT, 0, MPI_COMM_WORLD);

if(rank == (size - 1)) {
    elems = N - elems * (size - 1);
}

for(i=0; i < elems; i++) {
    tmp += f(v[i]);
}

MPI_Reduce(&tmp, &r, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

if(!rank) {
    printf("Result: %d\n", r);
}

MPI_Finalize();
```